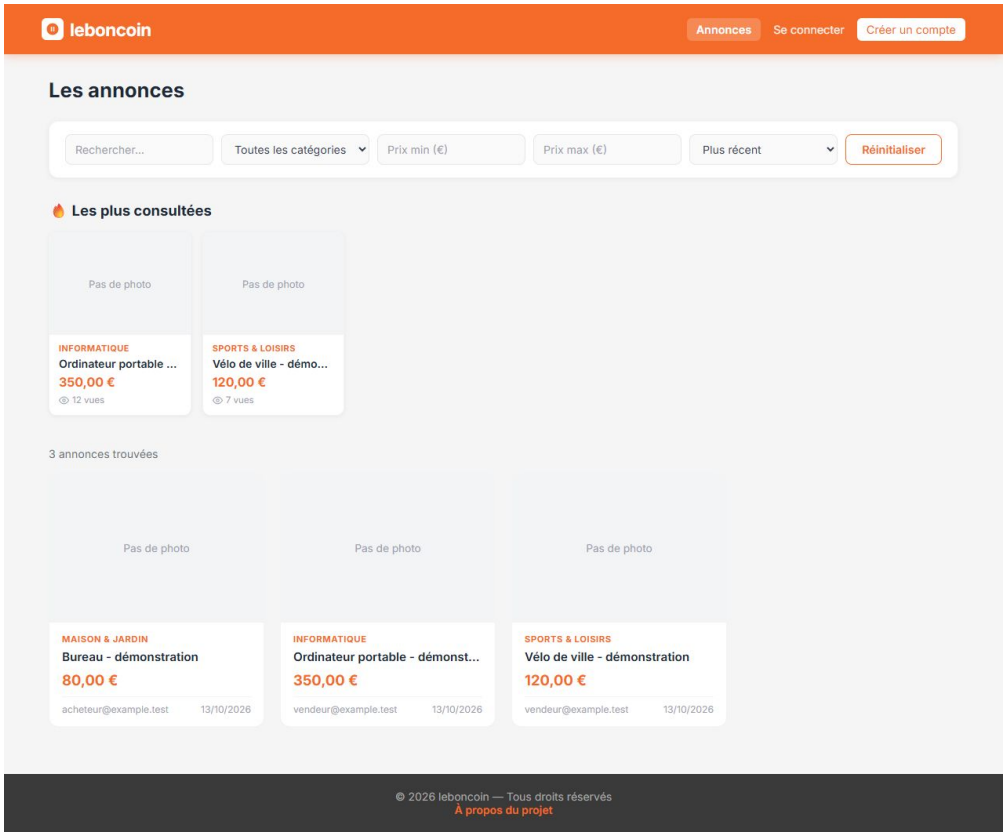
Leboncoin BTS SIO
Guide illustré des fonctionnalités

Des captures des véritables pages du projet, accompagnées des traitements PHP, SQL et JavaScript qui les font fonctionner.



Capture du projet exécuté en local. Annonces et comptes entièrement fictifs.

Un travail réalisé en groupe

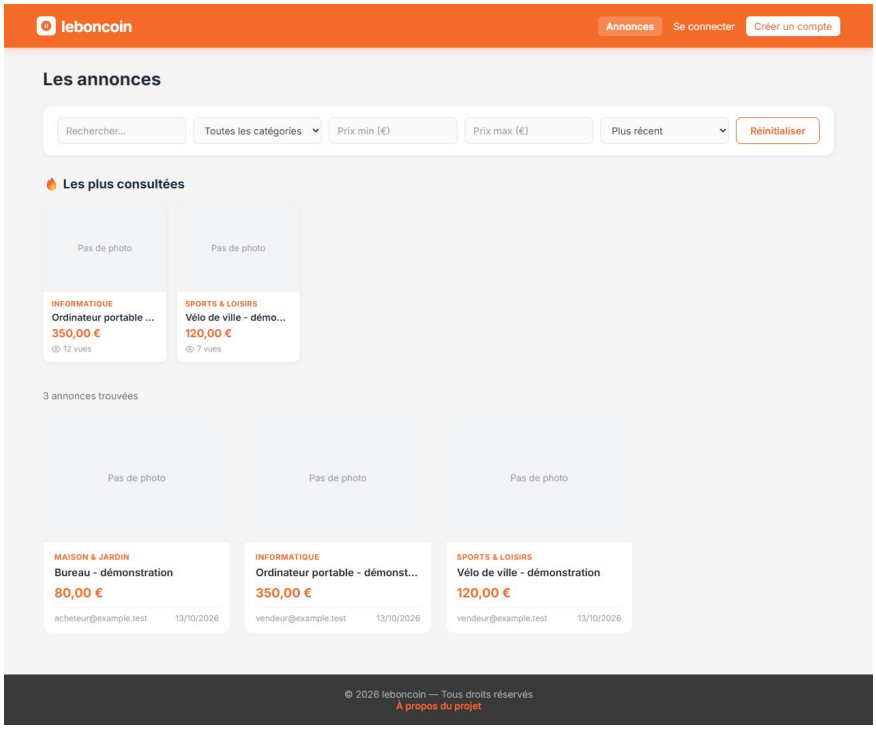
Le dépôt **patriciablbn/leboncoin-bts-sio** affiche les contributeurs **roman-corobca**, **patriciablbn** et **linamansour4106**. Ce guide décrit le code collectif ; il ne répartit pas les tâches. La contribution personnelle de Roman reste à compléter.

Origine et conditions des captures

Source : branche main, commit `e4a6829ec47a91fccc812797a5cb56320950b5f7`. Captures réalisées le 13 octobre 2026 sur une copie locale, avec PHP 8.3.28 et MariaDB 11.4.9. La configuration seule utilise une base de démonstration isolée ; les vues, contrôleurs, modèles et styles conservent le code source.

Les essais présentés ici documentent cette session de démonstration. Ils ne constituent pas des résultats de tests réalisés par Roman pendant le projet ni un audit complet. Aucune donnée réelle du groupe n'est utilisée.

01. Parcourir les annonces



Catalogue, cartes et annonces les plus consultées.

De l’écran au code

La route **annonces** de index.php appelle AnnonceController::getAll() et getPopulaires(5). AnnonceModel interroge la base ; annonces.view.php parcourt les résultats et affiche titre, catégorie, prix, vendeur et date. La section populaire trie les annonces par nb_vues décroissant.

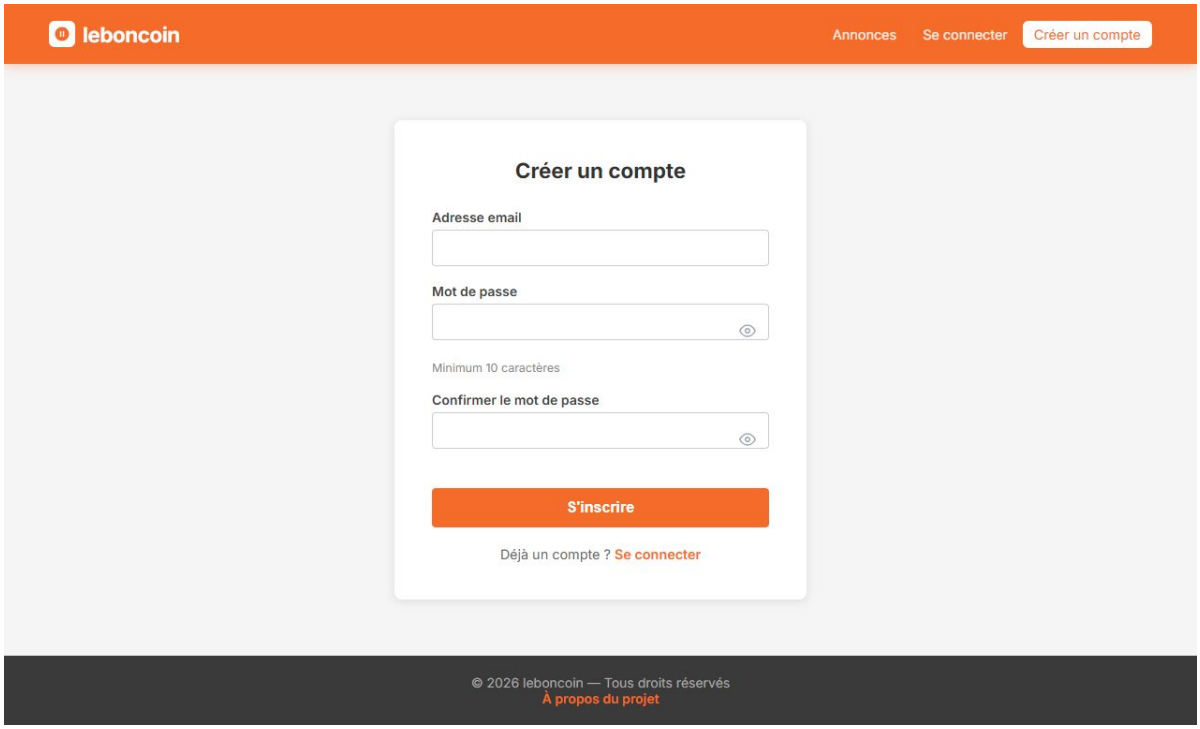
Extrait réel : models/annonce.model.php - lignes 56 à 64.

```
56      public function findPopulaires(int $limite = 5): array
57      {
58          $stmt = $this->pdo->prepare(
59              'SELECT a.*, u.email as vendeur_email
60               FROM annonces a
61               JOIN users u ON a.user_id = u.id
62               WHERE a.nb_vues > 0
63               ORDER BY a.nb_vues DESC
64               LIMIT ?'
```

Les liens de chaque carte transmettent l’identifiant de l’annonce. Les images viennent du dossier uploads/ ; lorsqu’aucune photo n’est enregistrée, la vue affiche « Pas de photo ».

Validation locale : Le catalogue a affiché les trois annonces initiales de la base fictive.

02. Créer un compte



Formulaire d'inscription : email et deux saisies du mot de passe.

De l'écran au code

Le formulaire envoie une requête POST à la route register. UserController::inscription() contrôle le format de l'email, une longueur minimale de 10 caractères, la confirmation et l'unicité. UserModel::create() insère l'email et le hash en base, puis une redirection conduit vers la connexion.

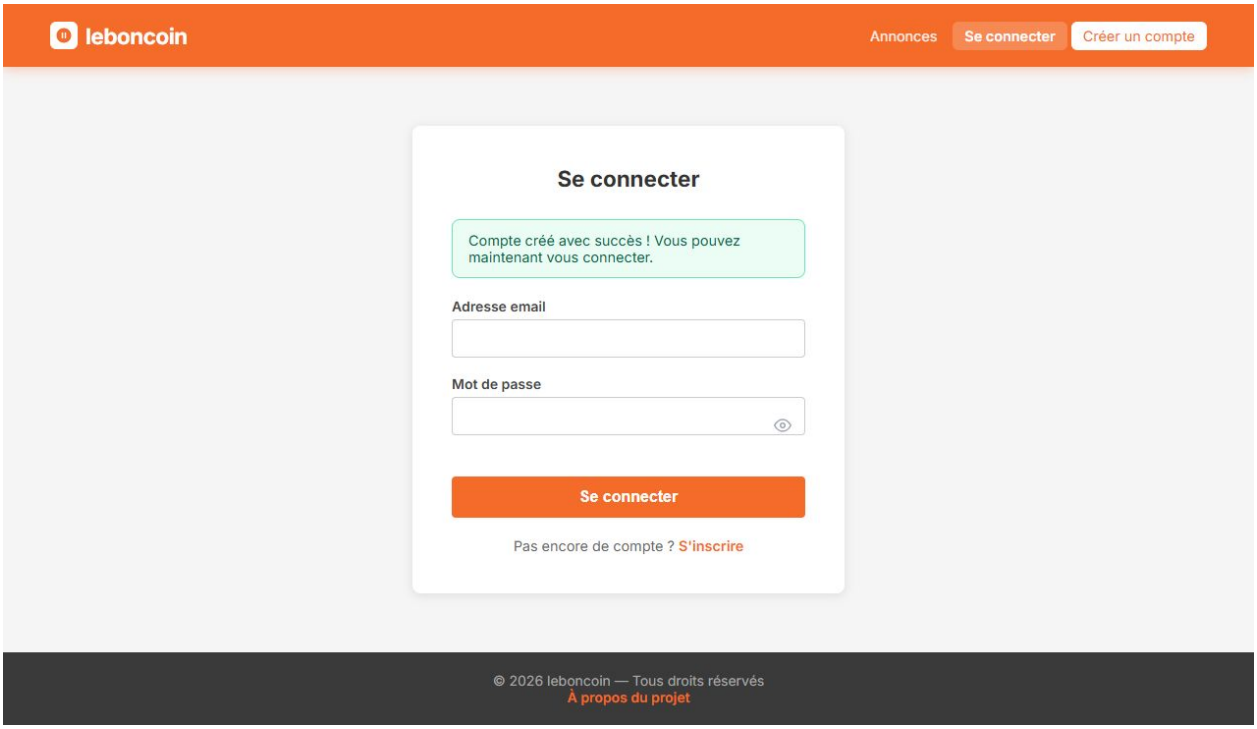
Extrait réel : controllers/user.controller.php - lignes 25 à 33.

```
25          if (strlen($password) < 10) {
26              return 'Le mot de passe doit contenir au moins 10 caractères.';
27          }
28          if ($password !== $confirm) {
29              return 'Les mots de passe ne correspondent pas.';
30          }
31          if ($this->model->emailExists($email)) {
32              return 'Cet email est déjà utilisé.';
33          }
```

La fonction password_hash() transforme le mot de passe avant stockage. L'icône œil est pilotée par .toggle-password dans assets/js/script.js : elle modifie le type du champ, sans remplacer les contrôles PHP.

Validation locale : Un compte visiteur@example.test a été créé ; le site a affiché le message de succès.

03. Se connecter et se déconnecter



Page de connexion après la création du compte de test.

De l’écran au code

UserController::connexion() cherche le compte par email via UserModel::findByEmail(). password_verify() compare le mot de passe au hash. En cas de succès, user_id et email sont placés dans la session ; le menu connecté présente Favoris, Messages et Mon compte.

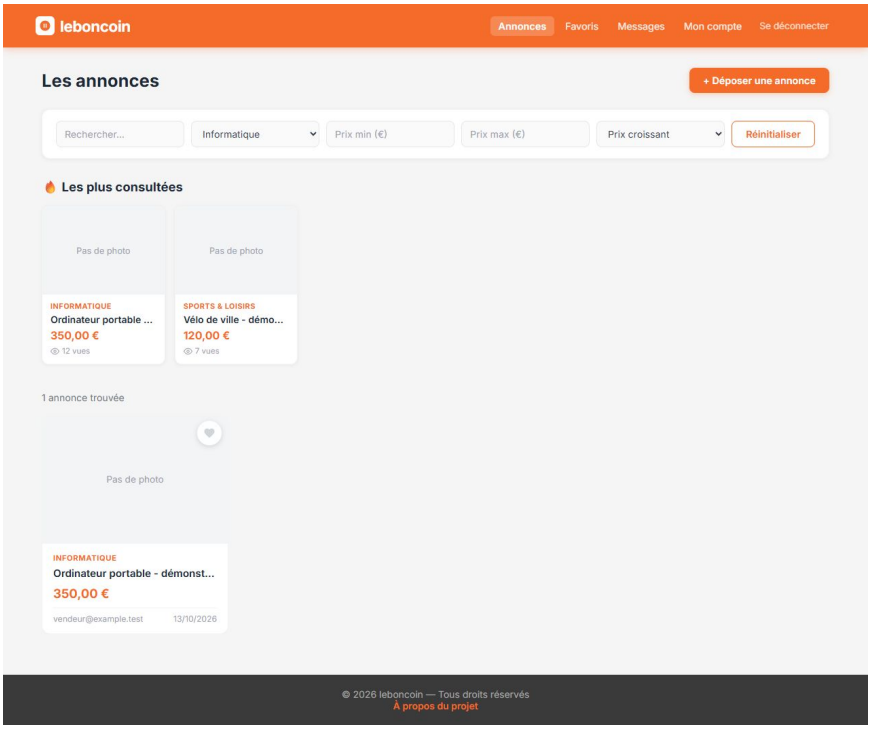
Extrait réel : controllers/user.controller.php - lignes 61 à 66.

```
61          $_SESSION['user_id'] = $user['id'];
62          $_SESSION['email']   = $user['email'];
63
64          header('Location: index.php');
65          exit;
66      }
```

La route action=logout appelle deconnexion() : elle vide la session, expire son cookie, puis appelle session_destroy(). La déconnexion est décrite d’après le code ; elle n’a pas été exécutée dans cette session de capture.

Validation locale : Le compte acheteur@example.test s’est connecté et les liens réservés à la session ont été affichés.

04. Rechercher, filtrer et trier



Chargement côté serveur avec catégorie Informatique et tri par prix croissant.

De l’écran au code

index.php récupère recherche, categorie, prix_min, prix_max et tri depuis GET. AnnonceModel::findAll() construit la clause WHERE et les paramètres PDO. Le tri utilise une liste autorisée et une correspondance match ; la recherche textuelle applique LIKE au titre et à la description.

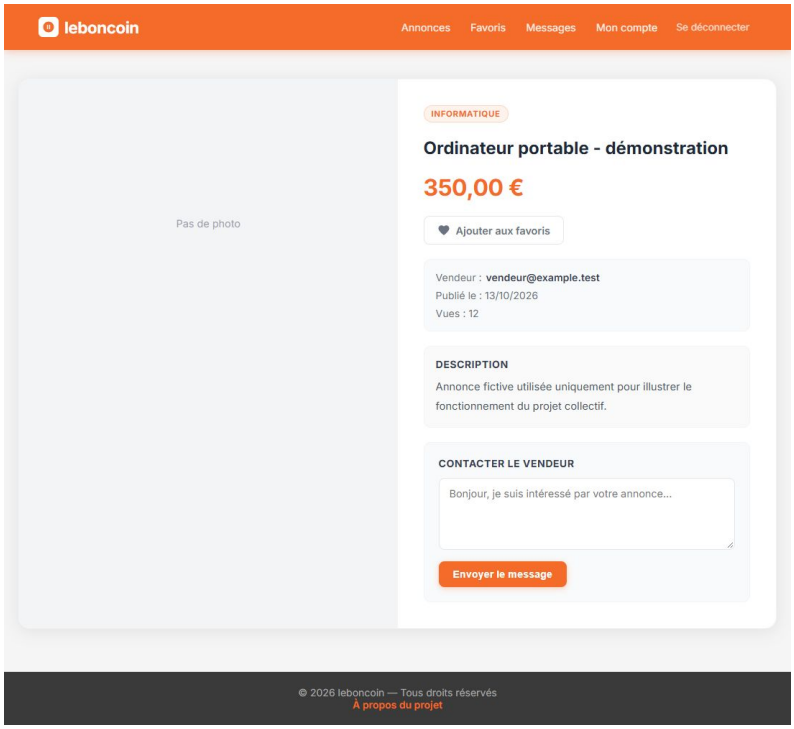
Extrait réel : models/annonce.model.php - lignes 19 à 27.

```
19          if (!empty($filtres['categorie'])) {
20              $sql .= ' AND a.categorie = ?';
21              $params[] = $filtres['categorie'];
22          }
23          if (!empty($filtres['prix_min'])) {
24              $sql .= ' AND a.prix >= ?';
25              $params[] = (float) $filtres['prix_min'];
26          }
27          if (!empty($filtres['prix_max'])) {
```

Cette capture correspond au traitement PHP par chargement de page. Elle ne prouve pas le fonctionnement AJAX : sa limite est expliquée à la fin du guide.

Validation locale : Une URL avec categorie=Informatique a renvoyé une seule annonce. Les autres combinaisons de prix et de recherche ne sont pas déclarées testées.

05. Consulter une annonce



Titre, prix, vendeur, description et formulaire pour contacter le vendeur.

De l’écran au code

Le paramètre GET id est converti en entier puis transmis à AnnonceController::getById(). Le modèle joint annonces à users pour obtenir le vendeur. Le contrôleur incrémente nb_vues après la lecture. annonce-detail.view.php affiche les champs et adapte les actions selon la session et l’auteur.

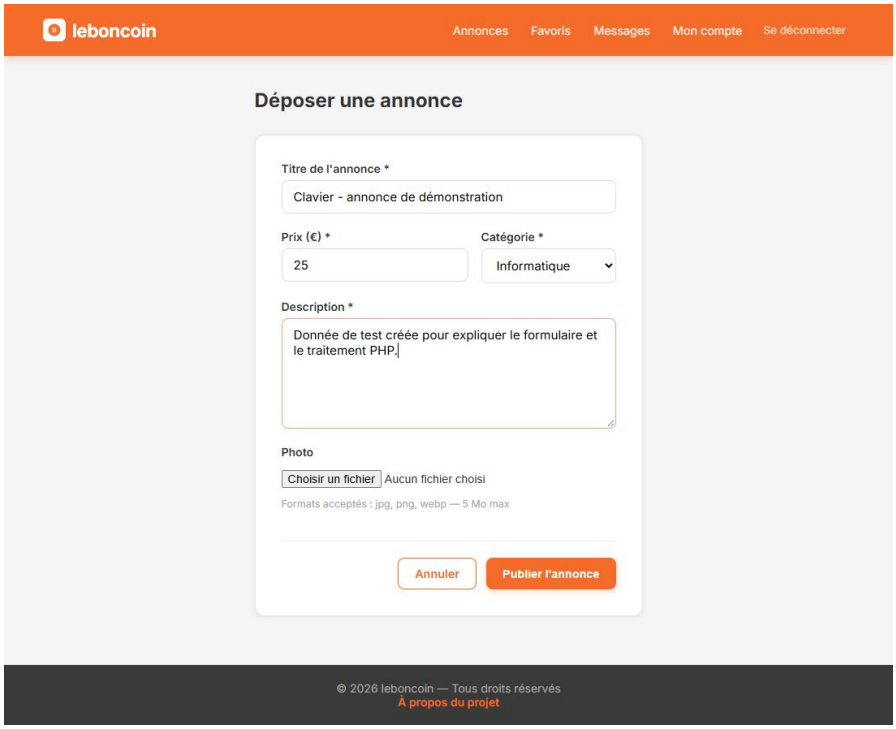
Extrait réel : controllers/annonce.controller.php - lignes 36 à 43.

```
36      public function getById(int $id): array|false
37      {
38          $annonce = $this->model->findById($id);
39          if ($annonce) {
40              $this->model->incrementVues($id);
41          }
42          return $annonce;
43      }
```

htmlspecialchars() échappe les valeurs affichées ; nl2br() conserve les retours à la ligne de la description. Le compteur montré provient de la lecture avant incrémentation, ce qui peut donner un décalage d’une vue sur la fiche.

Validation locale : La fiche de l’annonce 101 s’est affichée et le compteur a augmenté lors des consultations.

06. Déposer une annonce et sa photo



Formulaire rempli avec une annonce fictive, avant publication.

De l’écran au code

Le POST annonce_create est dispatché vers AnnonceController::creer(). Il exige une session, les champs obligatoires, un prix numérique non négatif et une catégorie autorisée. uploadPhoto() traite la photo facultative. AnnonceModel::create() insère l’annonce avec l’identifiant de l’utilisateur connecté.

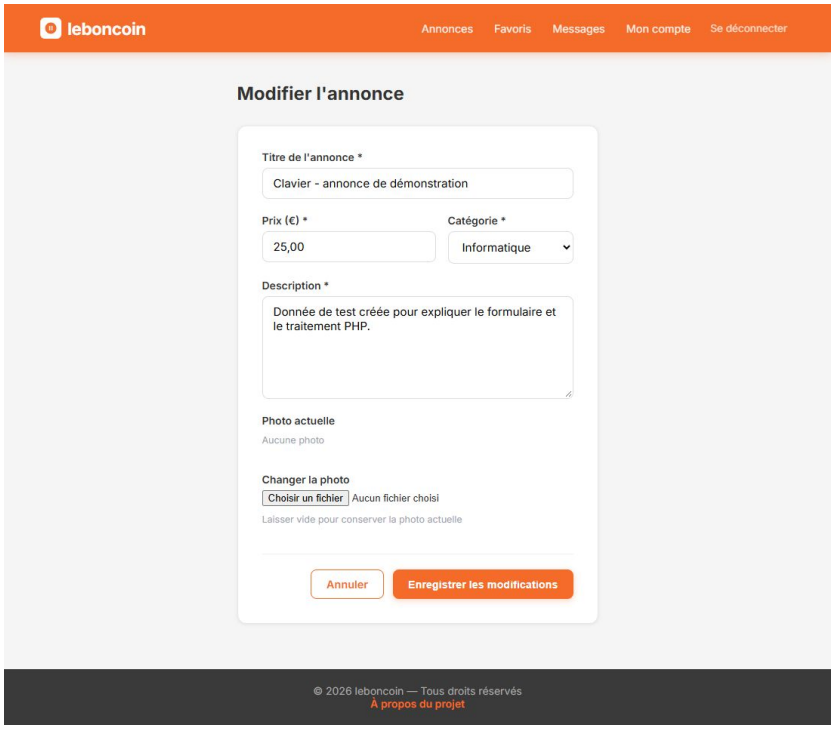
Extrait réel : controllers/annonce.controller.php - lignes 150 à 159.

```
150         $autorise  = ['jpg', 'jpeg', 'png', 'webp'];
151
152         if (!in_array($extension, $autorise)) {
153             return ['error' => 'Format de photo non autorisé (jpg, jpeg, png, webp).'];
154         }
155         if ($file['size'] > 5 * 1024 * 1024) {
156             return ['error' => 'La photo ne doit pas dépasser 5 Mo.'];
157         }
158
159         $nomFichier = uniqid('annonce_') . '.' . $extension;
```

Pour la photo, le code accepte jpg/jpeg/png/webp, impose 5 Mo maximum, produit un nom avec uniqid() et appelle move_uploaded_file(). Le téléversement n’a pas été testé ; la vérification du MIME et le résultat du déplacement restent à renforcer.

Validation locale : L’annonce fictive « Clavier » a été publiée sans photo et a porté le catalogue de trois à quatre annonces.

07. Modifier et supprimer ses annonces



Formulaire de modification de l’annonce du compte de démonstration.

De l’écran au code

Les routes annonce_edit et annonce_delete passent par le contrôleur. Avant la mutation, modifier() et supprimer() retrouvent l’annonce et vérifient que son user_id correspond à la session. Le modèle exécute UPDATE ou DELETE ; la suppression retire aussi la photo si elle existe.

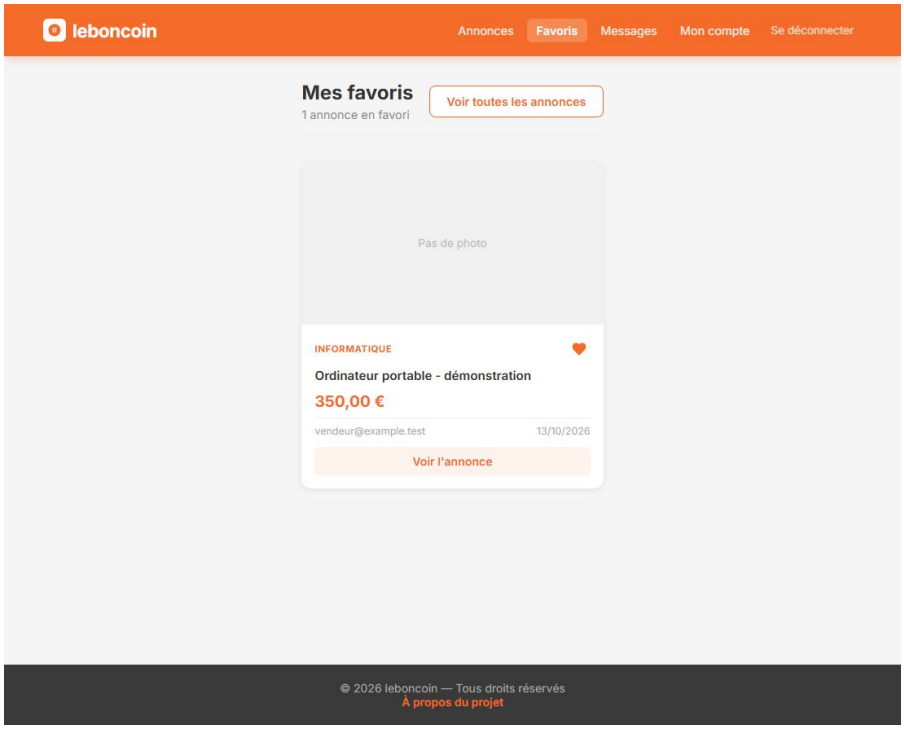
Extrait réel : controllers/annonce.controller.php - lignes 92 à 94.

```
92          if (!$annonce || $annonce['user_id'] !== (int) $_SESSION['user_id']) {
93              return ['error' => 'Annonce introuvable ou accès refusé.'];
94          }
```

La vue préremplit les champs. Le POST porte annonce_id ; la validation serveur demeure nécessaire même si les boutons ne sont visibles que pour le propriétaire. La suppression utilise une boîte de confirmation JavaScript, qui n’est pas une protection CSRF.

Validation locale : Le prix du Clavier a été modifié de 25 à 30 euros et le catalogue a montré la nouvelle valeur. La suppression n’a pas été exécutée.

08. Conserver des favoris



Une annonce enregistrée dans les favoris du compte de test.

De l’écran au code

Le bouton cœur soumet favori_toggle avec annonce_id. FavoriController::toggle() vérifie la session, teste l’existence du lien puis appelle add() ou remove(). La table favoris lie user_id et annonce_id ; sa contrainte unique interdit deux occurrences de la même paire.

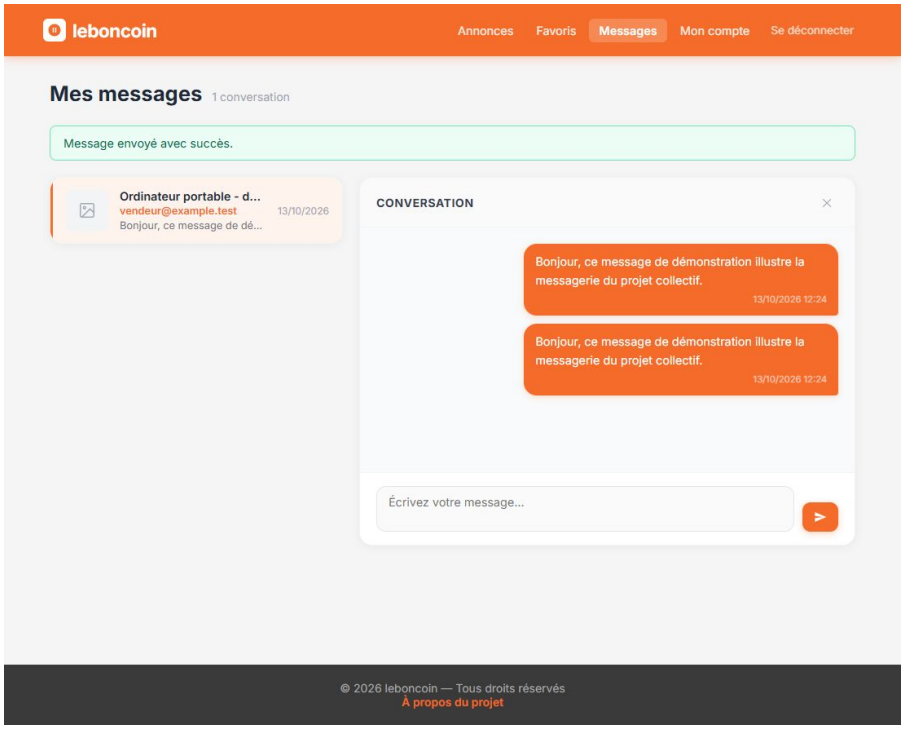
Extrait réel : controllers/favori.controller.php - lignes 23 à 28.

```
23          if ($this->model->isFavori($userId, $annonceId)) {
24              $this->model->remove($userId, $annonceId);
25          } else {
26              $this->model->add($userId, $annonceId);
27          }
28      }
```

FavoriModel::findByUser() joint favoris, annonces et users pour afficher les informations du vendeur. L’état est conservé en base, pas seulement dans le navigateur.

Validation locale : L’annonce Ordinateur a été ajoutée aux favoris ; la page a affiché une annonce et le bouton est devenu « Retirer des favoris ». Le retrait n’a pas été exécuté.

09. Envoyer et consulter des messages



Conversation avec vendeur@example.test ; échanges fictifs.

De l’écran au code

Le formulaire envoie message_send, annonce_id, destinataire_id et contenu. MessageController::envoyer() exige une session, un contenu non vide, une annonce existante et un destinataire différent de soi. MessageModel::create() enregistre l’échange et une redirection ouvre la conversation.

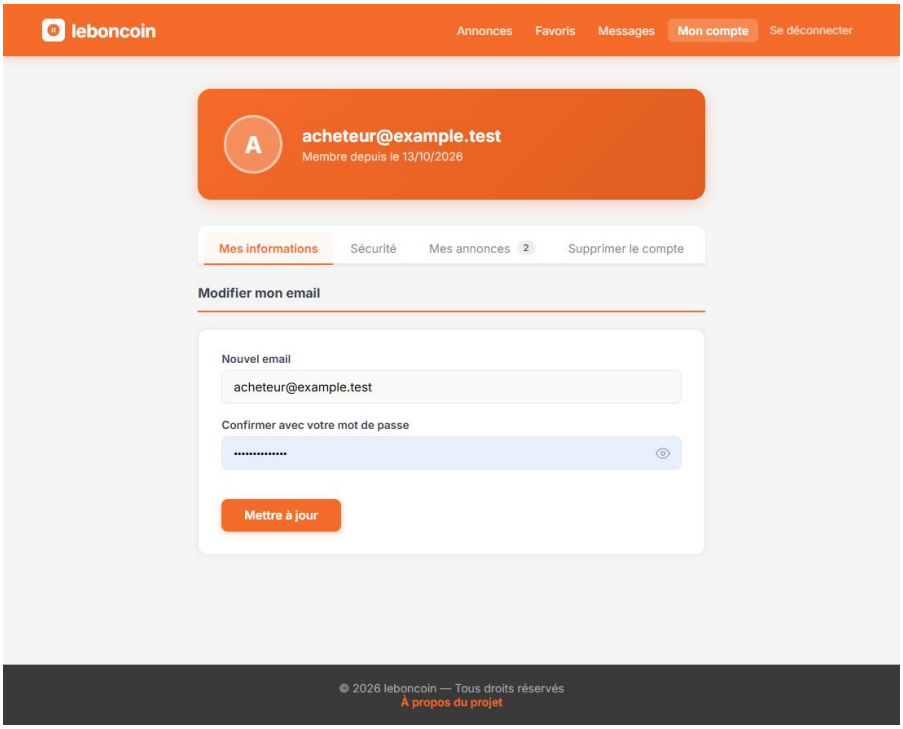
Extrait réel : models/message.model.php - lignes 55 à 63.

```
55      public function create(int $annonceId, int $expediteurId, int $destinataireId, string
    $contenu): bool
56      {
57          $stmt = $this->pdo->prepare('
58              INSERT INTO messages (annonce_id, expediteur_id, destinataire_id, contenu)
59              VALUES (?, ?, ?, ?)
60          ');
61          return $stmt->execute([$annonceId, $expediteurId, $destinataireId, $contenu]);
62      }
63
```

findByConversation() récupère les échanges dans les deux sens et les trie par date. getMessages() appelle markAsRead(). countUnread() alimente le badge du menu. Le destinataire soumis doit aussi être contrôlé côté serveur par rapport au contexte de l’annonce.

Validation locale : Un message a été envoyé et s’est affiché dans la conversation. Les textes et comptes sont ceux de la démonstration.

10. Gérer son profil



Onglet « Mes informations » et formulaire de modification de l'email.

De l'écran au code

La route profil exige une session. UserController::getProfil() charge le compte ; getMesAnnonces() charge ses annonces. modifierEmail() valide l'email, vérifie le mot de passe courant et contrôle les doublons avant UPDATE puis actualisation de l'email en session.

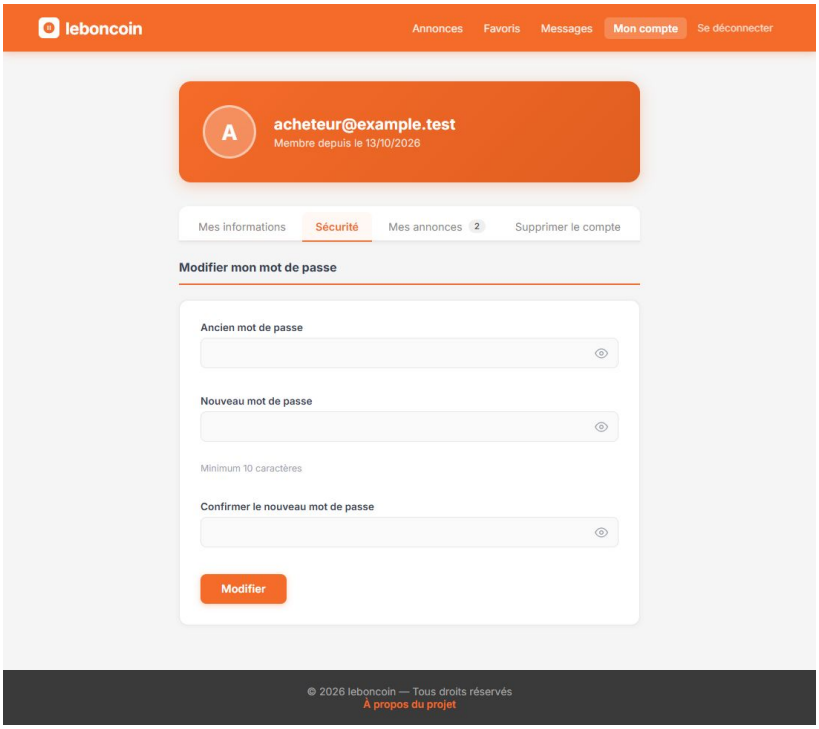
Extrait réel : controllers/user.controller.php - lignes 105 à 112.

```
105         $user = $this->model->findById((int) $_SESSION['user_id']);
106         if (!$user || !password_verify($password, $user['password'])) {
107             return 'Mot de passe incorrect.';
108         }
109
110         if ($this->model->emailExists($nouvelEmail)) {
111             return 'Cet email est déjà utilisé.';
112         }
```

Le JavaScript des .profil-tab retire puis ajoute la classe active aux onglets et panneaux. Ce changement d'affichage se déroule sans rechargement ; les modifications persistantes passent, elles, par PHP et la base.

Validation locale : La page du compte et ses onglets ont été affichés. La modification de l'email n'a pas été exécutée.

11. Sécurité et suppression du compte



Onglet Sécurité : ancien mot de passe, nouveau mot de passe et confirmation.

De l’écran au code

modifierMotDePasse() contrôle longueur et confirmation, vérifie l’ancien mot de passe puis enregistre un nouveau hash. supprimerCompte() demande le mot de passe courant, supprime le compte et détruit la session ; les clés étrangères ON DELETE CASCADE retirent les données liées.

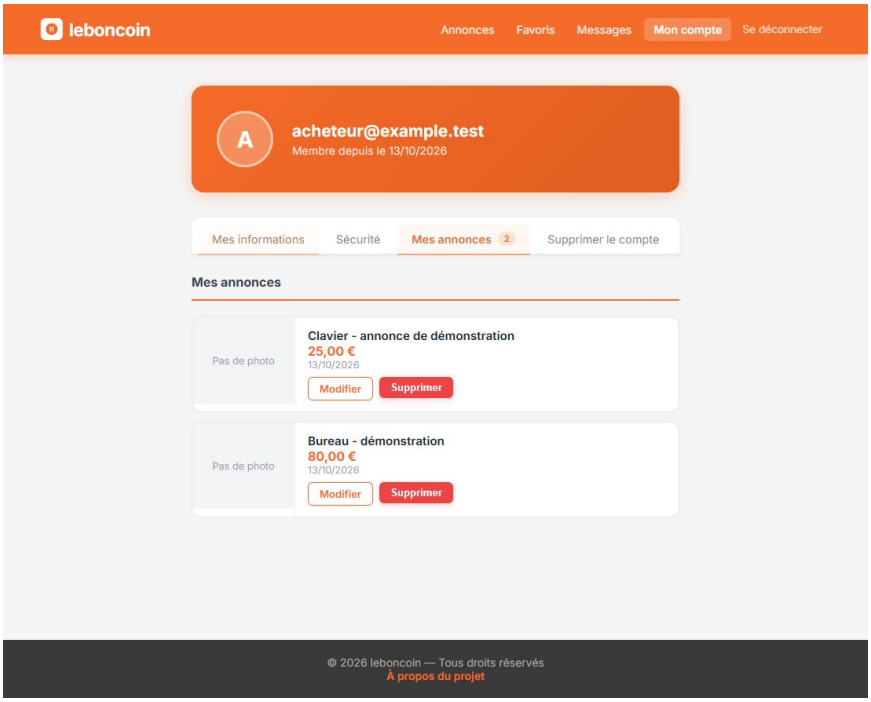
Extrait réel : controllers/user.controller.php - lignes 119 à 127.

```
119     public function modifierMotDePasse(): string
120     {
121         $ancien  = $_POST['ancien_password']  ?? '';
122         $nouveau = $_POST['nouveau_password'] ?? '';
123         $confirm = $_POST['confirm_password'] ?? '';
124
125         if (strlen($nouveau) < 10) {
126             return 'Le nouveau mot de passe doit contenir au moins 10 caractères.';
127         }
```

Le bouton œil facilite la saisie, mais ne renforce pas l’authentification. Une protection CSRF, une limitation des tentatives et une régénération d’identifiant de session doivent être évaluées avant mise en production.

Validation locale : Les formulaires Sécurité et Suppression ont été ouverts uniquement. Aucun changement de mot de passe ni suppression de compte n’est déclaré testé.

12. Navigation, présentation et données



Onglet Mes annonces : cartes personnelles et actions Modifier / Supprimer.

Organisation du code

Navigateur → index.php → contrôleur → modèle PDO → MariaDB → vue PHP → navigateur. index.php est le point d'entrée. views/components/header.php adapte le menu à la session ; le footer fournit le lien À propos. Les feuilles CSS sont sélectionnées selon la page.

Extrait réel : index.php - lignes 224 à 232.

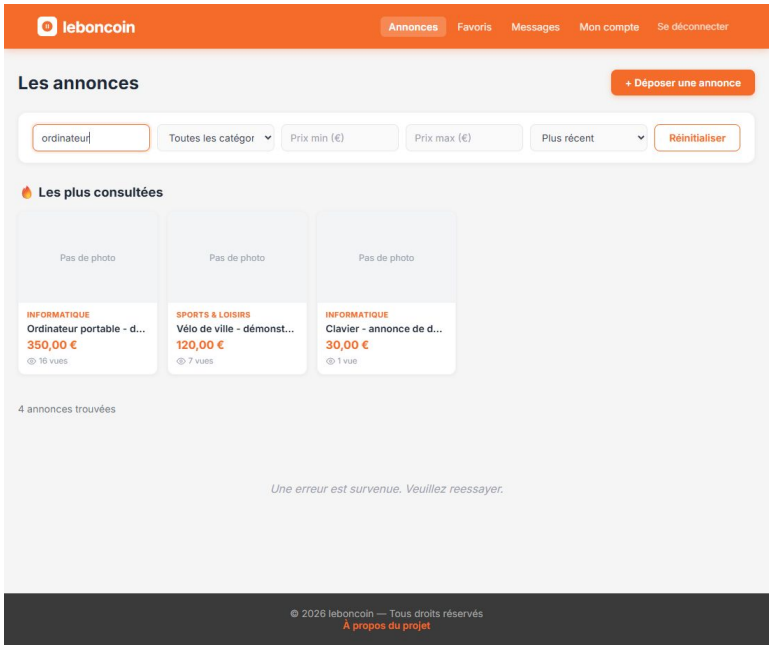
```
224  } elseif ($page === 'profil') {
225      if (empty($_SESSION['user_id'])) {
226          header('Location: index.php?page=login');
227          exit;
228      }
229      require_once __DIR__ . '/controllers/user.controller.php';
230      $userController = new UserController();
231      $profilData  = $userController->getProfil((int) $_SESSION['user_id']);
232      $mesAnnonces = $userController->getMesAnnonces((int) $_SESSION['user_id']);
```

Relations de la base

users possède des **annonces**. **favoris** associe un utilisateur à une annonce. **messages** associe annonce, expéditeur et destinataire. Les identifiants, index, contraintes uniques et clés étrangères sont définis dans db/leboncoin.sql.

La page À propos contient une présentation rédigée par le groupe. Les rôles et noms qu'elle affiche ne sont pas repris comme répartition confirmée : le guide conserve l'attribution collective et demande des précisions sur la contribution de Roman.

13. Limites observées et bilan des essais



Erreur réellement observée après la saisie d'un mot-clé dans le filtre dynamique.

Pourquoi le filtre dynamique échoue

assets/js/script.js masque le bouton Filtrer et intercepte les changements de champs. fetchAnnonces() demande du JSON à api/annonces.php. Ce fichier est absent de la branche main examinée ; la copie locale retourne 404, puis le code affiche « Une erreur est survenue ». Le filtrage par GET reste disponible par chargement direct de page.

Extrait réel : assets/js/script.js - lignes 99 à 101.

```
 99          const base = window.location.pathname.replace(/\/index.php$/, '').replace(/\/$/,
     '');
100          return `${base}/api/annonces.php?${params.toString()}`;
101      }
```

Ce que cette session a vérifié

Affichage du catalogue, inscription fictive, connexion, filtre serveur par catégorie, fiche détail et compteur, création sans photo, modification du prix, ajout d'un favori, envoi d'un message et navigation entre les onglets du profil.

Ce qui reste à tester

Téléversement, retraits et suppressions, changements d'email et de mot de passe, déconnexion, droits avec plusieurs comptes, toutes les combinaisons de filtres, comportements sur mobile et protections de sécurité. Les essais partiels ne prouvent pas que tout le projet est validé.

Configuration de démonstration

La connexion utilise des variables d'environnement, PDO natif avec ATTR_EMULATE_PREPARES=false comme dans la configuration source, et une base isolée. output_buffering=4096 permet les redirections malgré le HTML envoyé en début de index.php. Il est préférable de traiter les actions avant tout HTML.

14. Reproduire et présenter le projet

Installer la copie

• Préparer PHP 8 avec PDO MySQL et mbstring, puis MySQL ou MariaDB.

• Créer une base vide et importer uniquement db/leboncoin.sql.

• Définir DB_HOST, DB_PORT, DB_NAME, DB_USER et DB_PASSWORD pour la configuration de la copie.

• Autoriser l'écriture dans uploads/ ; servir le projet avec PHP. GitHub Pages ne peut pas exécuter ce code PHP.

• Créer des comptes et des annonces fictifs ; vérifier les parcours et noter chaque résultat.

Pour la soutenance

Présenter le besoin, montrer un écran, expliquer la route et le contrôleur, retrouver la requête du modèle puis décrire un test réel. Pour chaque tâche personnelle, ajouter les fichiers concernés, les commits, les difficultés et les solutions. Ne pas s'attribuer l'ensemble du projet.

Sources et traçabilité

Dépôt collectif : github.com/patriciablbn/leboncoin-bts-sio (accès privé). Tous les extraits numérotés proviennent du commit indiqué en introduction. Les captures proviennent de sa copie locale de démonstration ; elles n'ont pas été recréées sous forme de maquettes.

Fiche et téléchargements dans le portfolio public. Le groupe a autorisé la publication de la copie et du guide. Le dépôt d'origine reste inchangé.

Guide rédigé pour Roman COROBCA, contributeur du projet collectif. Les résultats de cette démonstration sont distincts des travaux personnels qui seront documentés ultérieurement.